

Explicación de clases - PioPioFly

1. Introducción

PioPioFly es un asistente de desplazamientos para aficionados de la UD Las Palmas. Su objetivo es combinar tres tipos de información:

- partidos fuera de casa obtenidos desde LaLiga,
- vuelos obtenidos desde AENA,
- enlace oficial de compra de entradas para desplazamientos.

El proyecto sigue una arquitectura **Lambda simplificada**:

- los **feeders** capturan datos y publican eventos en ActiveMQ,
- el **Event Store Builder** consume esos eventos y los guarda como histórico en ficheros `.events`,
- la **Business Unit** reconstruye o actualiza un **datamart SQLite** a partir del histórico y del consumo en vivo,
- la **API REST** y el **dashboard web** consultan ese datamart para ofrecer una vista útil al usuario final.

La idea importante para la defensa es esta: el sistema no solo captura datos, sino que separa claramente **fuentes**, **eventos**, **almacenamiento histórico**, **modelo de consulta** e **interfaz**.

2. Vista general de módulos

Módulo 'domain'

- Responsabilidad principal: definir modelos comunes, contratos y utilidades compartidas.
- Clases más importantes: `Match`, `FlightInfo`, `EventMessage`, `EventTopics`, `EventPublisher`, `ActiveMqEventPublisher`, `AirportMapping`.
- Relación con otros módulos: todos los demás módulos dependen de 'domain'.

Módulo 'app'

- Responsabilidad principal: orquestar la ejecución principal del sistema.
- Clases más importantes: `Main`, `DatabaseInitializer`.
- Relación con otros módulos: usa `matches-source`, `flights-source` y `domain`.

Módulo 'matches-source'

- Responsabilidad principal: capturar partidos fuera de casa de la UD Las Palmas desde LaLiga.
- Clases más importantes: `LaligaMatchScraper`, `AwayMatchService`, `AwayMatchRepository`.
- Relación con otros módulos: usa `domain` y persiste en `pio_pio_fly.db`.

Módulo 'flights-source'

- Responsabilidad principal: capturar vuelos desde AENA para los desplazamientos.
- Clases mÆs importantes: 'AenaFlightScraper', 'FlightInfoService', 'FlightInfoRepository'.
- Relaci3n con otros m3dulos: usa 'domain', lee partidos ya guardados y publica eventos 'FlightInfo'.

M3dulo 'event-store-builder'

- Responsabilidad principal: consumir eventos de ActiveMQ y almacenarlos en un event store en disco.
- Clases mÆs importantes: 'EventStoreBuilderApp', 'ActiveMqEventStoreSubscriber', 'EventStoreWriter'.
- Relaci3n con otros m3dulos: depende de 'domain' para topics y publisher comœn, y entrega hist3ricos a 'business-unit'.

M3dulo 'business-unit'

- Responsabilidad principal: construir la vista de negocio del sistema.
- Clases mÆs importantes: 'DatamartRepository', 'EventStoreDatamartLoader', 'BusinessUnitEventSubscriber', 'BusinessUnitWebServer', 'BusinessUnitApp'.
- Relaci3n con otros m3dulos: consume el hist3rico del event store, consume eventos en vivo desde ActiveMQ y expone una interfaz web local.

3. Explicaci3n clase por clase

3.1. M3dulo 'app'

Main.java

- Paquete: 'org.ulpgc.dacd.app'
- M3dulo: 'app'
- Responsabilidad: punto de entrada principal del sistema.
- Qu3 problema resuelve: centraliza en un œnico flujo la inicializaci3n de SQLite, la captura de partidos, la captura de vuelos y la publicaci3n de eventos.
- M3todos principales:
 - 'main(String[] args)'
- Dependencias/colaboradores:
 - 'DatabaseInitializer'
 - 'OneboxTicketLinkProvider'
 - 'ActiveMqEventPublisher'
 - 'AwayMatchService'
 - 'FlightInfoService'
 - 'LaligaMatchScraper'
 - 'AenaFlightScraper'
- C3mo encaja en el flujo general:
 - arranca la base local,
 - ejecuta la Fuente 1 de partidos,
 - muestra el enlace oficial de entradas,

- ejecuta la Fuente 2 de vuelos.
 - ¿Podría preguntar el profesor sobre esta clase:
- por qué el flujo es secuencial y no periódico,
- por qué el publisher de ActiveMQ se crea aquí,
- ¿ocurre si ActiveMQ no está disponible al arrancar.

DatabaseInitializer.java

- Paquete: 'org.ulpgc.dacd.app'
- Módulo: 'app'
- Responsabilidad: crear la base SQLite local del feeder principal.
- ¿qué problema resuelve: evita depender de una base creada manualmente y garantiza que existan las tablas al ejecutar 'Main'.
- Métodos principales:
- 'initialize()'
 - Dependencias/colaboradores:
- 'DatabaseConfig'
- JDBC/SQLite
 - Cómo encaja en el flujo general:
- prepara 'away_matches', 'flight_offers' y 'flight_infos',
- crea el índice único para evitar duplicados lógicos de vuelos.
 - ¿Podría preguntar el profesor sobre esta clase:
- por qué existe una tabla 'flight_offers' aunque no se use en Sprint 1,
- por qué se usa un índice único en 'flight_infos',
- por qué SQLite es suficiente para la fase local.

3.2. Módulo 'domain'

Match.java

- Paquete: 'org.ulpgc.dacd.domain'
- Módulo: 'domain'
- Responsabilidad: representar un partido.
- ¿qué problema resuelve: encapsula todos los atributos necesarios para mover un partido entre scraping, repositorio, eventos y datamart.
- Métodos principales:
- constructor vacío,
- constructor completo,
- getters,
- 'toString()'.
 - Dependencias/colaboradores:
- 'AwayMatchService'
- 'AwayMatchRepository'
- 'LaligaMatchScraper'
- 'EventStoreDatamartLoader'
 - Cómo encaja en el flujo general:
- es el modelo base de la Fuente 1.
 - ¿Podría preguntar el profesor sobre esta clase:
- por qué 'matchDate' y 'capturedAt' usan 'LocalDateTime',
- por qué 'destinationAirport' se guarda ya en el partido.

FlightInfo.java

- Paquete: 'org.ulpgc.dacd.domain'
- Módulo: 'domain'
- Responsabilidad: representar un vuelo capturado desde AENA.
- QuØ problema resuelve: unifica los datos mínimos necesarios para persistir, publicar y consultar vuelos.
- Métodos principales:
 - constructor vacío,
 - constructor completo,
 - getters,
 - 'toString()'.
- Dependencias/colaboradores:
 - 'AenaFlightScraper'
 - 'FlightInfoService'
 - 'FlightInfoRepository'
- Cómo encaja en el flujo general:
- es el modelo base de la Fuente 2 y del topic 'FlightInfo'.
 - QuØ podría preguntar el profesor sobre esta clase:
- por quØ 'scheduledDateTime' se almacena como 'String' y no como 'LocalDateTime',
- quØ campos forman la clave lógica de idempotencia.

FlightOffer.java

- Paquete: 'org.ulpgc.dacd.domain'
- Módulo: 'domain'
- Responsabilidad: representar una futura oferta de vuelo con precio.
- QuØ problema resuelve: deja preparada una entidad para una evolución posterior del proyecto hacia recomendaciones con precios.
- Métodos principales:
 - constructores,
 - getters,
 - 'toString()'.
- Dependencias/colaboradores:
- actualmente no participa en el flujo principal.
 - Cómo encaja en el flujo general:
- es una pieza de extensión del dominio, no del flujo activo final.
 - QuØ podría preguntar el profesor sobre esta clase:
- por quØ existe si en la entrega final no se usan precios,
- cómo encajaría en un sprint posterior.

AirportMapping.java

- Paquete: 'org.ulpgc.dacd.domain'
- Módulo: 'domain'
- Responsabilidad: mapear equipos locales y alias a códigos IATA.
- QuØ problema resuelve: la fuente de partidos conoce el rival, no el aeropuerto; esta clase traduce equipo local a aeropuerto de destino.
- Métodos principales:
 - constructor por defecto,

- constructor con mapa externo,
- 'getAirportCode(...)',
- 'getAirportByLocation()'.
 - Dependencias/colaboradores:
- 'AwayMatchService'
 - Cómo encaja en el flujo general:
- convierte 'RC Deportivo' en 'LCG', 'UD Almería' en 'LEI', etc.
 - ¿QuØ podría preguntar el profesor sobre esta clase:
- cómo se normalizan tildes y mayúsculas,
- por quØ se permite búsqueda por 'contains',
- quØ limitaciones tiene un mapeo estático.

DatabaseConfig.java

- Paquete: 'org.ulpgc.dacd.domain'
- Módulo: 'domain'
- Responsabilidad: centralizar la configuración de la base SQLite principal.
- ¿QuØ problema resuelve: evita repetir la URL 'jdbc:sqlite:pio_pio_fly.db'.
- Métodos principales:
- no tiene métodos; solo constantes.
 - Dependencias/colaboradores:
- 'DatabaseInitializer'
- 'AwayMatchRepository'
- 'FlightInfoRepository'
- 'FlightInfoService'
 - Cómo encaja en el flujo general:
- sirve como punto único de configuración para la base del feeder.
 - ¿QuØ podría preguntar el profesor sobre esta clase:
- por quØ separar configuración en una clase simple,
- cómo cambiaría si hubiera perfiles o entornos.

EventMessage.java

- Paquete: 'org.ulpgc.dacd.domain'
- Módulo: 'domain'
- Responsabilidad: representar el sobre común de todos los eventos.
- ¿QuØ problema resuelve: unifica el formato 'ts + ss + payload'.
- Métodos principales:
- 'capturedNow(...)'
- 'fromInstant(...)'
- getters
- 'toString()'
 - Dependencias/colaboradores:
- 'ActiveMqEventPublisher'
- 'AwayMatchService'
- 'FlightInfoService'
- 'EventStoreManualPublisher'
 - Cómo encaja en el flujo general:
- es el contrato de eventos entre feeders, broker, event store y business unit.
 - ¿QuØ podría preguntar el profesor sobre esta clase:

- por qu  'ts' se genera en UTC,
- qu  significa 'ss',
- por qu  el payload es un 'Map<String, Object>'.

EventTopics.java

- Paquete: 'org.ulpgc.dacd.domain'
- M dulo: 'domain'
- Responsabilidad: definir los nombres oficiales de topics.
- Qu  problema resuelve: evita strings duplicados y errores tipogr ficos.
- M todos principales:
- no tiene; solo constantes 'AWAY_MATCH' y 'FLIGHT_INFO'.
- Dependencias/colaboradores:
- publicadores y subscribers JMS.
- C mo encaja en el flujo general:
- alinea todos los m dulos alrededor de los mismos topics.
- Qu  podr a preguntar el profesor sobre esta clase:
- por qu  es  til centralizar los nombres,
- qu  pasar a si cada m dulo usara un literal diferente.

EventPublisher.java

- Paquete: 'org.ulpgc.dacd.domain'
- M dulo: 'domain'
- Responsabilidad: definir la abstracci n de publicaci n de eventos.
- Qu  problema resuelve: desacopla los servicios de captura del mecanismo concreto de mensajer a.
- M todos principales:
- 'publish(String topicName, EventMessage eventMessage)'
- 'close()'
- Dependencias/colaboradores:
- implementado por 'ActiveMqEventPublisher'
- usado por 'AwayMatchService' y 'FlightInfoService'
- C mo encaja en el flujo general:
- introduce una peque a inversi n de dependencias en la arquitectura.
- Qu  podr a preguntar el profesor sobre esta clase:
- por qu  usar una interfaz y no instanciar ActiveMQ directamente en los servicios,
- qu  ventaja da para pruebas o futuras extensiones.

ActiveMqEventPublisher.java

- Paquete: 'org.ulpgc.dacd.domain'
- M dulo: 'domain'
- Responsabilidad: publicar eventos JSON en ActiveMQ.
- Qu  problema resuelve: encapsula la conexi n JMS, la serializaci n y el env o persistente a topics.
- M todos principales:
- constructores con broker por defecto o configurable,
- 'publish(...)',
- 'close()'.

- Dependencias/colaboradores:
- 'EventPublisher'
- 'EventMessage'
- Jackson
- ActiveMQ JMS
 - Cómo encaja en el flujo general:
- es la salida común de eventos para los feeders y para el manual publisher.
 - ¿Podría preguntar el profesor sobre esta clase:
- por qué 'DeliveryMode.PERSISTENT',
- cómo se cierran recursos JMS,
- ¿ocurre si ActiveMQ no está disponible.

ResidentDiscountCalculator.java

- Paquete: 'org.ulpgc.dacd.domain'
- Módulo: 'domain'
- Responsabilidad: calcular un precio estimado con descuento de residente.
- ¿Qué problema resuelve: encapsula la regla del factor de descuento en una clase de dominio.
- Métodos principales:
 - constructor por defecto con factor '0.25',
 - constructor configurable,
 - 'estimateResidentPrice(...)'.
 - Dependencias/colaboradores:
- de momento no participa en el flujo final.
 - Cómo encaja en el flujo general:
- es una pieza de dominio preparada para una evolución futura orientada a precios.
 - ¿Podría preguntar el profesor sobre esta clase:
- por qué está en 'domain',
- cómo cambiaría si el descuento no fuera fijo.

3.3. Módulo 'matches-source'

MatchClient.java

- Paquete: 'org.ulpgc.dacd.matches'
- Módulo: 'matches-source'
- Responsabilidad: definir el contrato para obtener partidos.
- ¿Qué problema resuelve: desacopla el servicio de captura de la implementación concreta del scraping.
- Métodos principales:
 - 'fetchMatches()'
 - Dependencias/colaboradores:
- implementado por 'LaligaMatchScraper'
- usado por 'AwayMatchService'
 - Cómo encaja en el flujo general:
- permite que el servicio trabaje con una abstracción de fuente.
 - ¿Podría preguntar el profesor sobre esta clase:
- por qué una interfaz si solo hay una implementación,

- ¿qué ventaja tendría añadir otra fuente de partidos.

LaligaMatchScraper.java

- Paquete: 'org.ulpgc.dacd.matches'
- Módulo: 'matches-source'
- Responsabilidad: scrapear la página de próximos partidos de la UD Las Palmas en LaLiga.
- ¿qué problema resuelve: convierte HTML real en objetos 'Match'.
- Métodos principales:
 - 'fetchMatches()'
- métodos internos de parseo de filas, fecha, competición y equipos.
 - Dependencias/colaboradores:
- 'MatchClient'
- 'Match'
- JSoup
 - Cómo encaja en el flujo general:
- es la implementación real de la Fuente 1.
 - ¿qué podría preguntar el profesor sobre esta clase:
- por qué se eligió scraping y no API oficial,
- cómo se detectan filas válidas,
- ¿qué limitaciones tiene si cambia el HTML de LaLiga.

AwayMatchRepository.java

- Paquete: 'org.ulpgc.dacd.matches'
- Módulo: 'matches-source'
- Responsabilidad: persistir partidos fuera de casa en SQLite.
- ¿qué problema resuelve: aísla el acceso JDBC a 'away_matches'.
- Métodos principales:
 - 'save(Match match)'
- Dependencias/colaboradores:
- 'DatabaseConfig'
- JDBC
- 'Match'
 - Cómo encaja en el flujo general:
- guarda los resultados procesados por 'AwayMatchService'.
 - ¿qué podría preguntar el profesor sobre esta clase:
- por qué el repositorio no conoce ActiveMQ,
- cómo se transforma un 'Match' en columnas SQLite.

AwayMatchService.java

- Paquete: 'org.ulpgc.dacd.matches'
- Módulo: 'matches-source'
- Responsabilidad: coordinar la captura de partidos fuera de casa.
- ¿qué problema resuelve: concentra la lógica de negocio de la Fuente 1, no solo el scraping.
- Métodos principales:
 - 'captureAwayMatches()'
 - 'isAwayMatchForUdLasPalmas(...)'

- 'resolveDestinationAirport(...)'
- 'publishAwayMatchEvent(...)'
 - Dependencias/colaboradores:
- 'MatchClient'
- 'AirportMapping'
- 'AwayMatchRepository'
- 'EventPublisher'
 - Cómo encaja en el flujo general:
- filtra los partidos en los que la UD Las Palmas actúa como visitante,
- asigna el aeropuerto de destino,
- guarda el partido,
- publica evento 'AwayMatch'.
 - ¿Podría preguntar el profesor sobre esta clase:
- por qué esta lógica no está dentro del scraper,
- cómo se reconoce que la UD Las Palmas juega fuera,
- ¿qué ocurre si no se encuentra aeropuerto para el equipo local.

OneboxTicketLinkProvider.java

- Paquete: 'org.ulpgc.dacd.matches'
- Módulo: 'matches-source'
- Responsabilidad: exponer el enlace oficial de entradas de desplazamientos.
- ¿Qué problema resuelve: desacopla el enlace oficial del resto del flujo.
- Métodos principales:
- 'getOfficialAwayTicketUrl()'
 - Dependencias/colaboradores:
- 'Main'
 - Cómo encaja en el flujo general:
- aporta el enlace oficial al usuario sin persistirlo en base de datos.
 - ¿Podría preguntar el profesor sobre esta clase:
- por qué no se guarda en SQLite,
- por qué se aíslan una clase específica.

3.4. Módulo 'flights-source'

FlightInfoScraper.java

- Paquete: 'org.ulpgc.dacd.flights'
- Módulo: 'flights-source'
- Responsabilidad: definir el contrato de captura de vuelos.
- ¿Qué problema resuelve: desacopla 'FlightInfoService' de la implementación concreta de AENA.
- Métodos principales:
- 'fetchFlights(String originAirport, String destinationAirport, String date)'
 - Dependencias/colaboradores:
- implementado por 'AenaFlightScraper'
- usado por 'FlightInfoService'
 - Cómo encaja en el flujo general:
- permite que el servicio de negocio hable con una abstracción, no con HTTP directamente.

- QuØ podría preguntar el profesor sobre esta clase:
- por quØ una interfaz en lugar de usar directamente el scraper.

AenaFlightScraper.java

- Paquete: 'org.ulpgc.dacd.flights'
- M dulo: 'flights-source'
- Responsabilidad: consultar AENA y convertir la respuesta en objetos 'FlightInfo'.
- QuØ problema resuelve: encapsula la complejidad t cnica del endpoint 'AENA_ConsultarVuelos', el parseo JSON y los fallos TLS.
- M todos principales:
- 'fetchFlights(...)'
- 'scrapeFlights(...)'
- 'fetchEndpointFlights(...)'
- 'fetchFlightsFromAirport(...)'
- 'fetchAenaEndpointBody(...)'
- 'fetchAenaEndpointBodyWithSystemCurl(...)'
- parseos y filtros internos por ruta y fecha.
 - Dependencias/colaboradores:
- 'FlightInfoScraper'
- 'FlightInfo'
- Jackson
- 'HttpClient'
- 'curl' del sistema como fallback remoto real
 - C mo encaja en el flujo general:
- es la implementaci n real de la Fuente 2.
- consulta salidas o llegadas de AENA,
- filtra por origen, destino y fecha exacta,
- devuelve solo vuelos v lidos para el desplazamiento.
 - QuØ podr a preguntar el profesor sobre esta clase:
- c mo se consulta AENA,
- por quØ se usa POST vac o,
- por quØ existe fallback con 'curl',
- c mo se filtran vuelos por origen, destino y fecha,
- c mo se evita usar un fallback local antiguo como si fuera dato real.

FlightInfoRepository.java

- Paquete: 'org.ulpgc.dacd.flights'
- M dulo: 'flights-source'
- Responsabilidad: guardar vuelos en 'flight_infos'.
- QuØ problema resuelve: aisla el acceso JDBC y la l gica de upsert de vuelos.
- M todos principales:
- 'save(FlightInfo flightInfo)'
 - Dependencias/colaboradores:
- 'DatabaseConfig'
- JDBC
- 'FlightInfo'
 - C mo encaja en el flujo general:

- persiste los vuelos capturados por AENA,
- evita duplicados con 'INSERT ... ON CONFLICT ... DO UPDATE'.
 - ¿QuØ podría preguntar el profesor sobre esta clase:
- ¿quØ campos forman la clave lØgica,
- por quØ se usa upsert y no insert simple.

FlightInfoService.java

- Paquete: 'org.ulpgc.dacd.flights'
- MØdulo: 'flights-source'
- Responsabilidad: coordinar la captura de vuelos en funciØn de los partidos guardados.
- ¿QuØ problema resuelve: transforma partidos fuera de casa en consultas concretas a AENA.
- MØtodos principales:
 - 'captureFlightsForAwayMatches()'
 - 'loadAwayMatches()'
 - 'buildFlightQueries(...)'
 - 'publishFlightInfoEvent(...)'
- Dependencias/colaboradores:
 - 'FlightInfoScraper'
 - 'FlightInfoRepository'
 - 'EventPublisher'
 - 'Match'
- ¿CØmo encaja en el flujo general:
- lee 'away_matches',
- genera consultas de ida y vuelta,
- captura vuelos,
- guarda en SQLite,
- publica eventos 'FlightInfo'.
 - ¿QuØ podría preguntar el profesor sobre esta clase:
- por quØ la lØgica de fechas no estÆ en la web sino aquØ,
- cØmo se calculan ida y vuelta,
- ¿quØ pasa si la fecha del partido no es vÆlida.

AenaLcgDebugRunner.java

- Paquete: 'org.ulpgc.dacd.flights'
- MØdulo: 'flights-source'
- Responsabilidad: ejecutar un diagnØstico acotado para el caso 'LPA -> LCG'.
- ¿QuØ problema resuelve: permite verificar el scraper sin insertar datos ni publicar eventos.
- MØtodos principales:
 - 'main(String[] args)'
- Dependencias/colaboradores:
 - 'AenaFlightScraper'
- ¿CØmo encaja en el flujo general:
- no forma parte del flujo de producciØn; es una utilidad de diagnØstico.
 - ¿QuØ podría preguntar el profesor sobre esta clase:
- por quØ fue Øtil para depurar AENA,

- por quØ no guarda en base de datos.

3.5. Mdulo 'event-store-builder'

EventStoreBuilderApp.java

- Paquete: 'org.ulpgc.dacd.eventstore'
- Mdulo: 'event-store-builder'
- Responsabilidad: arrancar el subscriber durable del event store.
- QuØ problema resuelve: expone un proceso dedicado y separado para construir el histrico de eventos.
- Mtodos principales:
- 'main(String[] args)'
 - Dependencias/colaboradores:
- 'ActiveMqEventStoreSubscriber'
- 'EventStoreWriter'
- 'EventTopics'
 - Cmo encaja en el flujo general:
- mantiene vivo el proceso que escucha ActiveMQ y escribe en disco.
 - QuØ podra preguntar el profesor sobre esta clase:
- por quØ estE separada de 'app',
- por quØ usa 'CountDownLatch' y 'shutdown hook'.

ActiveMqEventStoreSubscriber.java

- Paquete: 'org.ulpgc.dacd.eventstore'
- Mdulo: 'event-store-builder'
- Responsabilidad: suscribirse durablemente a los topics y reenviar eventos al escritor del event store.
- QuØ problema resuelve: encapsula reconexin JMS, suscripciones durables y tratamiento de mensajes.
- Mtodos principales:
- 'start()'
- 'close()'
- 'runSubscriber()'
- 'connectAndSubscribe()'
- 'handleMessage(...)'
- 'createDurableSubscription(...)'
 - Dependencias/colaboradores:
- ActiveMQ JMS
- 'EventStoreWriter'
 - Cmo encaja en el flujo general:
- escucha 'AwayMatch' y 'FlightInfo',
- recibe 'TextMessage',
- escribe el JSON original en el event store.
 - QuØ podra preguntar el profesor sobre esta clase:
- quØ es una suscripcin durable,
- por quØ usa un 'clientId' estable,
- cmo reacciona si ActiveMQ se cae.

EventStoreWriter.java

- Paquete: 'org.ulpgc.dacd.eventstore'
- Módulo: 'event-store-builder'
- Responsabilidad: persistir eventos consumidos en ficheros NDJSON.
- Qué problema resuelve: traduce un evento JSON en una línea dentro de 'eventstore/{topic}/{ss}/{YYYYMMDD}.events'.
- Métodos principales:
- 'append(String topic, String eventJson)'
- 'dayFromTimestamp(...)'
- 'toJsonLine(...)'
- Dependencias/colaboradores:
- Jackson
- 'Files'/NIO
- Cómo encaja en el flujo general:
- es el escritor del histórico del sistema.
- Qué podría preguntar el profesor sobre esta clase:
- por qué se usa NDJSON,
- cómo se calcula el nombre diario a partir de 'ts' en UTC,
- por qué se hace 'append' y no sobrescritura.

EventStoreManualPublisher.java

- Paquete: 'org.ulpgc.dacd.eventstore'
- Módulo: 'event-store-builder'
- Responsabilidad: publicar eventos manuales de prueba.
- Qué problema resuelve: facilita probar ActiveMQ y el subscriber sin depender de LaLiga o AENA.
- Métodos principales:
- 'main(String[] args)'
- 'publishAwayMatchEvent(...)'
- 'publishFlightInfoEvent(...)'
- Dependencias/colaboradores:
- 'ActiveMqEventPublisher'
- 'EventMessage'
- 'EventTopics'
- Cómo encaja en el flujo general:
- es una herramienta auxiliar de demo y diagnóstico.
- Qué podría preguntar el profesor sobre esta clase:
- por qué los datos manuales se excluyen del dashboard público,
- por qué sigue siendo útil aunque no sea parte del flujo final.

3.6. Módulo 'business-unit'

BusinessUnitConfig.java

- Paquete: 'org.ulpgc.dacd.business'
- Módulo: 'business-unit'
- Responsabilidad: centralizar la configuración de la unidad de negocio.
- Qué problema resuelve: evita dispersar broker, client ID, base SQLite, ruta del event store y puerto web.
- Métodos principales:

- no tiene; solo constantes.
 - Dependencias/colaboradores:
- 'BusinessUnitApp'
- 'DatamartInitializer'
- 'BusinessUnitWebServer'
- 'EventStoreDatamartLoader'
 - Cómo encaja en el flujo general:
- es el punto comœen de configuración del módulo.
 - QuØ podría preguntar el profesor sobre esta clase:
- por quØ hay una base distinta ('business_unit.db') separada de 'pio_pio_fly.db'.

BusinessUnitApp.java

- Paquete: 'org.ulpgc.dacd.business'
- Módulo: 'business-unit'
- Responsabilidad: arrancar la unidad de negocio.
- QuØ problema resuelve: coordina la preparación del datamart, la recarga histórica, la sincronización en vivo y la web.
- Métodos principales:
- 'main(String[] args)'
 - Dependencias/colaboradores:
- 'DatamartInitializer'
- 'DatamartRepository'
- 'EventStoreDatamartLoader'
- 'BusinessUnitEventSubscriber'
- 'BusinessUnitWebServer'
 - Cómo encaja en el flujo general:
- inicializa el datamart,
- recarga históricos desde el event store,
- inicia la sincronización en vivo,
- arranca la web en 'localhost:8080'.
 - QuØ podría preguntar el profesor sobre esta clase:
- por quØ la recarga histórica se hace al arranque,
- quØ pasa si ActiveMQ estÆ caído,
- por quØ la interfaz pœblica principal es la web y no la CLI.

BusinessUnitCli.java

- Paquete: 'org.ulpgc.dacd.business'
- Módulo: 'business-unit'
- Responsabilidad: ofrecer una interfaz de consola interactiva.
- QuØ problema resuelve: proporciona una UI alternativa para consultar el datamart y probar el módulo sin navegador.
- Métodos principales:
- 'start()'
- 'showUpcomingAwayMatches()'
- 'showFlightsByDestination(...)'
- 'showTravelRecommendations()'
- 'showDatamartSummary()'

- 'reloadEventStoreHistory()'
- 'showBusinessUnitStatus()'
- Dependencias/colaboradores:
- 'DatamartRepository'
- 'EventStoreDatamartLoader'
- 'BusinessUnitEventSubscriber'
- Cómo encaja en el flujo general:
- es una interfaz secundaria conservada en el proyecto.
- ¿QuØ podría preguntar el profesor sobre esta clase:
- por quØ se mantuvo aunque la entrega final tenga web,
- cómo reutiliza el mismo repositorio del dashboard.

BusinessUnitWebServer.java

- Paquete: 'org.ulpgc.dacd.business'
- Módulo: 'business-unit'
- Responsabilidad: exponer API REST y dashboard web local.
- ¿QuØ problema resuelve: convierte el datamart en una interfaz usable por el usuario final.
- Métodos principales:
- 'start()'
- 'stop()'
- handlers de '/', '/api/status', '/api/summary', '/api/matches', '/api/next-trip', '/api/destinations', '/api/flights', '/api/recommendations', '/api/reload-eventstore', '/api/live-sync/start', '/api/live-sync/stop'
- métodos auxiliares 'sendJson(...)', 'sendHtml(...)', 'sendError(...)', parseo de query y lectura del logo.
- Dependencias/colaboradores:
- 'DatamartRepository'
- 'EventStoreDatamartLoader'
- 'BusinessUnitEventSubscriber'
- 'HttpServer'
- Jackson
- Cómo encaja en el flujo general:
- es la capa final de presentación del proyecto.
- ¿QuØ podría preguntar el profesor sobre esta clase:
- por quØ se usó 'HttpServer' de Java y no Spring,
- cómo se separa API interna y dashboard público,
- cómo se calculan y muestran los vuelos de ida y vuelta del siguiente desplazamiento.

BusinessUnitEventSubscriber.java

- Paquete: 'org.ulpgc.dacd.business'
- Módulo: 'business-unit'
- Responsabilidad: consumir eventos de ActiveMQ y actualizar el datamart en vivo.
- ¿QuØ problema resuelve: da capacidad de refresco en tiempo real sin depender solo de recargas históricas.

- Métodos principales:
- 'start()'
- 'stop()'
- 'isActive()'
- 'runSubscriber()'
- 'connect()'
- 'handleMessage(...)'
- Dependencias/colaboradores:
- ActiveMQ JMS
- Jackson
- 'DatamartRepository'
- Cómo encaja en el flujo general:
- escucha los mismos topics que el event store,
- actualiza 'business_unit.db' directamente con upserts.
 - ¿Qué podría preguntar el profesor sobre esta clase:
- cómo evita arrancar dos veces,
- cómo reconecta tras una caída,
- por qué la business unit puede seguir arrancando aunque el broker no esté disponible.

DatamartInitializer.java

- Paquete: 'org.ulpgc.dacd.business'
- Módulo: 'business-unit'
- Responsabilidad: crear la base 'business_unit.db'.
- ¿Qué problema resuelve: garantiza que el datamart exista con sus tablas e índices.
- Métodos principales:
- 'initialize()'
- Dependencias/colaboradores:
- 'BusinessUnitConfig'
- JDBC/SQLite
- Cómo encaja en el flujo general:
- prepara 'away_matches_datamart' y 'flight_infos_datamart'.
 - ¿Qué podría preguntar el profesor sobre esta clase:
- por qué el datamart es una base separada,
- qué índices únicos se usan para evitar duplicados.

DatamartRepository.java

- Paquete: 'org.ulpgc.dacd.business'
- Módulo: 'business-unit'
- Responsabilidad: centralizar toda la persistencia y consulta del datamart.
- ¿Qué problema resuelve: concentra en una sola clase el acceso JDBC de negocio.
- Métodos principales:
- 'getSummary()'
- 'saveAwayMatchFromEvent(...)'
- 'saveFlightInfoFromEvent(...)'
- 'findUpcomingAwayMatches()'
- 'findFlightsByDestination(...)'

- 'findAvailableDestinations()'
- 'buildTravelRecommendations()'
- 'findNextAwayMatchWithFlights()'
 - Dependencias/colaboradores:
- 'DatamartSummary'
- 'AwayMatchView'
- 'FlightInfoView'
- 'TravelRecommendation'
- 'NextTripView'
 - Cómo encaja en el flujo general:
- es el núcleo de consulta de la business unit.
- también deduplica partidos y vuelos,
- excluye fuentes manuales del dashboard público,
- calcula ventanas temporales de ida y vuelta alrededor del partido.
 - ¿Qué podría preguntar el profesor sobre esta clase:
- cómo funciona el upsert del datamart,
- cómo se reconstruye el estado desde eventos,
- cómo se deduplican partidos y vuelos,
- cómo se elige el siguiente desplazamiento.

DatamartSummary.java

- Paquete: 'org.ulpgc.dacd.business'
- Módulo: 'business-unit'
- Responsabilidad: representar un resumen agregado del datamart.
- ¿Qué problema resuelve: agrupa métricas de negocio en una sola estructura simple.
- Métodos principales:
- al ser 'record', expone automáticamente sus componentes.
 - Dependencias/colaboradores:
- 'DatamartRepository'
- 'BusinessUnitCli'
- 'BusinessUnitWebServer'
 - Cómo encaja en el flujo general:
- sirve al resumen de estado y a la API REST.
 - ¿Qué podría preguntar el profesor sobre esta clase:
- por qué un 'record' es adecuado aquí,
- ¿qué métricas se consideran relevantes.

EventStoreDatamartLoader.java

- Paquete: 'org.ulpgc.dacd.business'
- Módulo: 'business-unit'
- Responsabilidad: recargar el datamart desde el event store histórico.
- ¿Qué problema resuelve: permite reconstruir o inicializar el datamart sin depender del broker en tiempo real.
- Métodos principales:
- 'load()'
- 'loadFile(...)'
- 'loadLine(...)'

- 'saveAwayMatch(...)'
- 'saveFlightInfo(...)'
 - Dependencias/colaboradores:
- Jackson
- 'DatamartRepository'
- 'DatamartLoadResult'
- 'EventTopics'
 - Cómo encaja en el flujo general:
- es la capa batch/histórica de la arquitectura Lambda simplificada.
 - QuØ podría preguntar el profesor sobre esta clase:
- cómo extrae el topic desde la ruta,
- quØ ocurre si faltan 'ts', 'ss' o 'payload',
- cómo decide si un evento se carga o se omite.

DatamartLoadResult.java

- Paquete: 'org.ulpgc.dacd.business'
- Módulo: 'business-unit'
- Responsabilidad: representar el resultado de una recarga histórica.
- QuØ problema resuelve: devuelve mØtricas claras del proceso de carga.
- MØtodos principales:
- al ser 'record', expone directamente sus componentes.
 - Dependencias/colaboradores:
- 'EventStoreDatamartLoader'
- 'BusinessUnitCli'
- 'BusinessUnitWebServer'
- 'BusinessUnitApp'
 - Cómo encaja en el flujo general:
- informa cuÆntos eventos se procesaron, cargaron, omitieron o fallaron.
 - QuØ podría preguntar el profesor sobre esta clase:
- por quØ es œtil devolver un resumen estructurado y no solo logs.

AwayMatchView.java

- Paquete: 'org.ulpgc.dacd.business'
- Módulo: 'business-unit'
- Responsabilidad: representar un partido listo para consulta en el datamart.
- QuØ problema resuelve: separa la vista de consulta de la entidad de captura original.
- MØtodos principales:
- componentes del 'record'.
 - Dependencias/colaboradores:
- 'DatamartRepository'
- 'BusinessUnitCli'
- 'BusinessUnitWebServer'
 - Cómo encaja en el flujo general:
- es la proyección de partidos usada por la capa de consulta.
 - QuØ podría preguntar el profesor sobre esta clase:
- por quØ usar una vista específica y no reutilizar 'Match'.

FlightInfoView.java

- Paquete: 'org.ulpgc.dacd.business'
- Módulo: 'business-unit'
- Responsabilidad: representar un vuelo listo para consulta.
- QuØ problema resuelve: desacopla la lectura del datamart de la entidad 'FlightInfo' original.
- Métodos principales:
- componentes del 'record'.
 - Dependencias/colaboradores:
- 'DatamartRepository'
- 'BusinessUnitCli'
- 'BusinessUnitWebServer'
 - Cómo encaja en el flujo general:
- es la proyección de vuelos usada por API y dashboard.
 - QuØ podría preguntar el profesor sobre esta clase:
- por quØ separar entidad y vista de lectura.

TravelRecommendation.java

- Paquete: 'org.ulpgc.dacd.business'
- Módulo: 'business-unit'
- Responsabilidad: representar una recomendación de desplazamiento.
- QuØ problema resuelve: empaqueta partido, destino, vuelos disponibles, vuelo sugerido, motivo y nivel.
- Métodos principales:
- componentes del 'record',
- enum 'RecommendationLevel'.
 - Dependencias/colaboradores:
- 'DatamartRepository'
- 'BusinessUnitCli'
 - Cómo encaja en el flujo general:
- modela una lógica de negocio derivada del datamart.
 - QuØ podría preguntar el profesor sobre esta clase:
- cómo se decide entre 'ALTA', 'MEDIA', 'BAJA' y 'SIN_VUELOS',
- por quØ esta recomendación es simple y no usa precios.

NextTripView.java

- Paquete: 'org.ulpgc.dacd.business'
- Módulo: 'business-unit'
- Responsabilidad: representar el siguiente desplazamiento con sus vuelos asociados.
- QuØ problema resuelve: da una cénica estructura para la pantalla principal del dashboard.
- Métodos principales:
- componentes del 'record'.
 - Dependencias/colaboradores:
- 'DatamartRepository'
- 'BusinessUnitWebServer'
 - Cómo encaja en el flujo general:
- combina el siguiente partido con vuelos de ida, vuelos de vuelta y ventana

temporal.

- ¿Qué podría preguntar el profesor sobre esta clase:
- por qué separar ida y vuelta,
- ¿qué significa 'invalidMatchDate'.

4. Flujo completo del sistema

1. 'Main' inicializa la base SQLite principal con 'DatabaseInitializer'.
2. 'AwayMatchService' usa 'LigaMatchScraper' para capturar próximos partidos.
3. 'AwayMatchService' filtra solo los partidos fuera de casa de la UD Las Palmas.
4. 'AirportMapping' asigna automáticamente el aeropuerto de destino a partir del equipo local.
5. 'AwayMatchRepository' guarda esos partidos en 'away_matches'.
6. 'AwayMatchService' publica eventos 'AwayMatch' mediante 'ActiveMqEventPublisher'.
7. 'FlightInfoService' lee los partidos guardados, calcula consultas de ida y vuelta y usa 'AenaFlightScraper'.
8. 'FlightInfoRepository' guarda o actualiza los vuelos en 'flight_infos'.
9. 'FlightInfoService' publica eventos 'FlightInfo' en ActiveMQ.
10. 'ActiveMqEventStoreSubscriber' consume ambos topics y 'EventStoreWriter' los guarda en 'eventstore/{topic}/{ss}/{YYYYMMDD}.events'.
11. 'BusinessUnitApp' arranca, crea el datamart con 'DatamartInitializer' y lanza una recarga histórica con 'EventStoreDatamartLoader'.
12. 'BusinessUnitEventSubscriber' se suscribe en vivo a ActiveMQ y actualiza el datamart conforme llegan nuevos eventos.
13. 'DatamartRepository' sirve las consultas de negocio sobre 'business_unit.db'.
14. 'BusinessUnitWebServer' expone API REST y dashboard web, que muestran el siguiente desplazamiento y sus vuelos asociados.

5. ¿Qué clases defender en profundidad

Main

- Es importante porque demuestra la orquestación completa del sistema.
- Permite explicar el orden de ejecución y por qué los módulos están desacoplados.

AwayMatchService

- Es el ejemplo más claro de servicio de aplicación.
- Combina fuente externa, regla de negocio, persistencia y publicación de eventos.

FlightInfoService

- Es clave porque transforma un partido en varias consultas reales a AENA.
- Ahí se entiende bien la lógica de ida y vuelta y el filtrado por fechas del desplazamiento.

AenaFlightScraper

- Es una de las clases más técnicas del proyecto.
- Permite defender HTTP, parseo JSON, filtrado, normalización y tratamiento de fallos reales.

ActiveMqEventPublisher

- Es importante para explicar la parte de Publisher/Subscriber.
- Justifica el uso de eventos persistentes y topics.

ActiveMqEventStoreSubscriber

- Es clave para explicar el Event Store Builder.
- Permite hablar de suscripción durable y reconexión.

EventStoreWriter

- Es la clase que materializa el patrón Event Store.
- Sirve para justificar el formato NDJSON y la organización por topic/fuente/fecha.

EventStoreDatamartLoader

- Es fundamental para explicar por qué la arquitectura es Lambda simplificada y no solo streaming.
- Permite reconstruir el datamart desde histórico.

DatamartRepository

- Es el corazón de la business unit.
- Aquí están los upserts, las consultas de negocio y la deduplicación.

BusinessUnitEventSubscriber

- Es importante porque conecta la capa de eventos en vivo con el datamart.
- Permite explicar robustez, reconexión y actualización continua.

BusinessUnitWebServer

- Es la clase más visible para la demo.
- Permite defender la interfaz final, la API REST y el dashboard público.

6. Preguntas típicas del profesor

¿Por qué arquitectura Lambda y no Kappa?

Porque el sistema combina dos caminos hacia el datamart:

- uno en vivo desde ActiveMQ,
- otro histórico desde el event store.

Eso encaja mejor con una Lambda simplificada que con una Kappa pura.

¿Qué es ActiveMQ?

Es el broker de mensajería del sistema. Recibe los eventos publicados por los

feeders y permite que otros módulos los consuman de forma desacoplada.

¿QuØ es un topic?

Es un canal de publicación/suscripción. En este proyecto hay dos: 'AwayMatch' y 'FlightInfo'.

¿QuØ es una suscripción durable?

Es una suscripción JMS que mantiene el estado aunque el subscriber se desconecte. Cuando vuelve a conectarse con el mismo 'clientId', puede recuperar mensajes pendientes.

¿QuØ es el event store?

Es el histórico de eventos guardado en disco. Aquí se almacena como NDJSON en ficheros '.events', organizados por topic, fuente y día.

¿QuØ es el datamart?

Es una base orientada a consulta, construida para responder rápido a la UI y a la API. En este proyecto es 'business_unit.db'.

¿Por quØ SQLite?

Porque el proyecto está pensado para ejecución local, demo y consultas rápidas sin depender de una infraestructura pesada de base de datos.

¿Cómo se evitan duplicados?

- En 'flight_infos' y 'flight_infos_datamart' mediante índices únicos y upsert.
- En la consulta pública, además, 'DatamartRepository' deduplica partidos y vuelos con claves lógicas normalizadas.

¿QuØ pasa si ActiveMQ se cae?

- Los feeders pueden seguir guardando en SQLite local.
- El Event Store Builder reintenta conexión.
- La Business Unit sigue pudiendo arrancar con el histórico del event store y reintenta la sincronización en vivo.

¿QuØ pasa si AENA falla?

No se inventan vuelos. El sistema muestra mensajes claros y, en la web, botones de búsqueda alternativa si no hay vuelos cargados.

¿Cómo se reconstruye el datamart?

'EventStoreDatamartLoader' recorre 'eventstore/', lee cada línea JSON y vuelve a aplicar upserts sobre 'business_unit.db'.

¿QuØ aporta la business-unit al usuario final?

Transforma datos técnicos de múltiples fuentes en una experiencia útil: siguiente partido fuera, vuelos cercanos, enlace oficial de entrada y dashboard

web claro.

7. Clases listadas y comprobación

Se han encontrado las clases pedidas en el proyecto:

- 'Main'
- 'DatabaseInitializer'
- 'Match'
- 'FlightInfo'
- 'AirportMapping'
- 'DatabaseConfig'
- 'EventMessage'
- 'EventTopics'
- 'EventPublisher'
- 'ActiveMqEventPublisher'
- 'AwayMatchService'
- 'AwayMatchRepository'
- 'LaligaMatchScraper'
- 'MatchClient'
- 'OneboxTicketLinkProvider'
- 'AenaFlightScraper'
- 'AenaLcgDebugRunner'
- 'FlightInfoService'
- 'FlightInfoRepository'
- 'EventStoreBuilderApp'
- 'ActiveMqEventStoreSubscriber'
- 'EventStoreWriter'
- 'EventStoreManualPublisher'
- 'BusinessUnitApp'
- 'BusinessUnitConfig'
- 'BusinessUnitCli'
- 'BusinessUnitWebServer'
- 'BusinessUnitEventSubscriber'
- 'DatamartInitializer'
- 'DatamartRepository'
- 'DatamartSummary'
- 'EventStoreDatamartLoader'
- 'DatamartLoadResult'
- 'AwayMatchView'
- 'FlightInfoView'
- 'TravelRecommendation'
- 'NextTripView'

Además, se han encontrado clases adicionales útiles para la defensa:

- 'FlightOffer'
- 'ResidentDiscountCalculator'
- 'FlightInfoScraper'

No se incluyen 'package-info.java' porque no contienen lógica de aplicación,

solo metadatos de paquete.